

TP3 - Application Multi-Conteneurs avec Docker

Auteur: Haytame EL ATRAoui
Auteur: Haytame EL ATRAoui
Module: Gnie Logiciel - ENSA-BM
Module: Gnie Logiciel - ENSA-BM
Professeur: Pr. BE.ELBAGHAZAoui
Professeur: Pr. BE.ELBAGHAZAoui
Date: 2026
Date: 2026

Description

Ce TP couvre la dockerisation d'une application web multi-conteneurs utilisant Nginx, PostgreSQL et PgAdmin. Il comprend :

Ce TP couvre la dockerisation d'une application web multi-conteneurs utilisant Nginx, PostgreSQL et PgAdmin. Il co

1. **Création de l'image Docker** pour le frontend (Nginx)
1. **Création de l'image Docker** pour le frontend (Nginx)
2. **Configuration de PostgreSQL** comme base de données
2. **Configuration de PostgreSQL** comme base de données
3. **Orchestration avec Docker Compose** des services
3. **Orchestration avec Docker Compose** des services
4. **Gestion des volumes** pour la persistance des données
4. **Gestion des volumes** pour la persistance des données
5. **Mise en réseau** entre les conteneurs
5. **Mise en réseau** entre les conteneurs

Objectifs Pdagogiques

Comprendre les bases de la containerisation Docker
Crer des images Docker personnalisées avec Dockerfile
Orchestrer plusieurs conteneurs avec Docker Compose
Gérer les volumes et la persistance des données
Configurer les réseaux Docker
Utiliser des variables d'environnement
Gérer les dépendances entre conteneurs

Structure du Projet

...
TP3_DOCKER/
TP3_DOCKER/
frontend/
frontend/

TP3 - Application Multi-Conteneurs avec Docker

```
Dockerfile      # Image personnalisée Nginx
Dockerfile      # Image personnalisée Nginx
index.html      # Page HTML servie par Nginx
index.html      # Page HTML servie par Nginx
.dockerignore   # Fichiers exclus de la build
.dockerignore   # Fichiers exclus de la build
backend/
backend/
db.env          # Variables d'environnement PostgreSQL
db.env          # Variables d'environnement PostgreSQL
docker-compose.yml # Orchestration des services
docker-compose.yml # Orchestration des services
README.md       # Ce fichier
README.md       # Ce fichier
...

```

Services Configurés

1. Frontend (Nginx)

```
**Port**: 8080 80
**Image**: nginx:alpine
**Fonction**: Serveur web statique
**URL d'accès**: http://localhost:8080

```

2. Database (PostgreSQL 13)

```
**Port**: 5432 (interne)
**Image**: postgres:13
**Variables d'environnement**:
POSTGRES_USER: admin
POSTGRES_PASSWORD: adminpassword
POSTGRES_DB: myapp
**Volume**: db-data (persistance)

```

3. PgAdmin (Administration DB)

```
**Port**: 5050 80
**Image**: dpage/pgadmin4:latest
**URL d'accès**: http://localhost:5050
**Identifiants par défaut**:
Email: admin@admin.com
Mot de passe: admin

```

Installation et Exécution

TP3 - Application Multi-Conteneurs avec Docker

Prerequis

Docker Desktop (ou Docker + Docker Compose installs)

Port 8080 disponible (frontend)

Port 5050 disponible (pgadmin)

Dmarrage de l'application

```
```bash
```

### Se placer dans le rpertoire TP3\_DOCKER

```
cd TP3_DOCKER
```

```
cd TP3_DOCKER
```

### Construire et lancer les conteneurs

```
docker-compose up --build
```

```
docker-compose up --build
```

### (Optionnel) Lancer en arrire-plan

```
docker-compose up -d --build
```

```
docker-compose up -d --build
```

```
```
```

Arrt de l'application

```
```bash
```

```
docker-compose down
```

```
docker-compose down
```

```
```
```

Arrt avec suppression des volumes

```
```bash
```

```
docker-compose down -v
```

```
docker-compose down -v
```

```
```
```

Vrification et Validation

1. Vrifier les conteneurs actifs

```
```bash
```

## TP3 - Application Multi-Conteneurs avec Docker

```
docker-compose ps
docker-compose ps
...
```

```
Sortie attendue:
Sortie attendue:
...
```

NAME	STATUS	PORTS
frontend	Up X seconds	0.0.0.0:8080->80/tcp
db	Up X seconds	5432/tcp
pgadmin	Up X seconds	0.0.0.0:5050->80/tcp

### 2. Accder au Frontend

Ouvrir un navigateur et aller : **http://localhost:8080**  
Vous devriez voir la page HTML du projet Dockeris

### 3. Accder PgAdmin

URL: **http://localhost:5050**  
Email: **admin@admin.com**  
Mot de passe: **admin**

### 4. Configurer la connexion PostgreSQL dans PgAdmin

Dans PgAdmin, cliquer sur "Add New Server"  
Onglet "General":  
Name: `docker-postgres`  
Onglet "Connection":  
Host name/address: `db` (nom du service Docker)  
Port: `5432`  
Username: `admin`  
Password: `adminpassword`  
Database: `myapp`  
Cliquer "Save"

## Concepts Docker Matriss

### Dockerfile

## TP3 - Application Multi-Conteneurs avec Docker

Utilisation d'images de base (nginx:alpine)

Instructions COPY, EXPOSE, CMD

Optimisation de la taille avec alpine

### Docker Compose

Définition de services multi-conteneurs

Gestion des ports (port mapping)

Configuration des volumes (persistance)

Configuration des réseaux (communication inter-conteneurs)

Variables d'environnement et fichiers .env

Dépendances entre services (depends\_on)

### Concepts Avancés

Volumes nommés pour la persistance

Réseaux personnalisés (bridge networks)

Stratégies de redémarrage (restart: unless-stopped)

Fichiers .dockerignore pour optimiser la build

### Réseau Docker

...

Réseau: app-network

Réseau: app-network

Frontend (Nginx) | Port 8080

Frontend (Nginx) | Port 8080

Database (PostgreSQL) | Port 5432

Database (PostgreSQL) | Port 5432

PgAdmin | Port 5050

PgAdmin | Port 5050

...

Communication interne:

Communication interne:

Frontend PostgreSQL: via `db:5432`

PgAdmin PostgreSQL: via `db:5432`

### Persistance des Données

Le volume `db-data` assure que :

Le volume `db-data` assure que :

Les données PostgreSQL persistent après l'arrêt du conteneur

Les données survivent au redémarrage de l'application

## TP3 - Application Multi-Conteneurs avec Docker

Pour rinitialiser les donnees : ``docker-compose down -v``

### Dpannage

#### Les conteneurs ne dmarrent pas

```
```bash
```

Vrifier les logs

```
docker-compose logs
docker-compose logs
```

Vrifier un conteneur spcifique

```
docker-compose logs frontend
docker-compose logs frontend
docker-compose logs db
docker-compose logs db
docker-compose logs pgadmin
docker-compose logs pgadmin
```
```

#### Impossible d'accder localhost:8080

```
```bash
```

Vrifier que le port n'est pas occup

```
netstat -ano | findstr :8080 # Windows
netstat -ano | findstr :8080 # Windows
lsof -i :8080                # macOS/Linux
lsof -i :8080                # macOS/Linux
```
```

#### Erreur de connexion PostgreSQL dans PgAdmin

- Vrifier que le service ``db`` est en cours d'excuton
- Vrifier les identifiants (user: admin, password: adminpassword)
- Vrifier que le host est bien ``db`` (pas localhost ou 127.0.0.1)

### Amliorations Possibles

1. Ajouter un backend API (Node.js, Flask, Django)
1. Ajouter un backend API (Node.js, Flask, Django)
2. Implmenter des healthchecks
2. Implmenter des healthchecks

## TP3 - Application Multi-Conteneurs avec Docker

3. Ajouter SSL/TLS pour le frontend
3. Ajouter SSL/TLS pour le frontend
4. Configurer une CI/CD avec GitHub Actions
4. Configurer une CI/CD avec GitHub Actions
5. Ajouter des tests automatisés
5. Ajouter des tests automatisés
6. Utiliser Docker Swarm ou Kubernetes pour l'orchestration
6. Utiliser Docker Swarm ou Kubernetes pour l'orchestration

### Ressources Utiles

[Documentation Docker](<https://docs.docker.com/>)

[Docker Compose Reference](<https://docs.docker.com/compose/compose-file/>)

[Nginx Documentation](<https://nginx.org/en/>)

[PostgreSQL Documentation](<https://www.postgresql.org/docs/>)

[PgAdmin Documentation](<https://www.pgadmin.org/docs/>)